

Article

An Improved Reference Paper Collection System Using Web Scrapping with Three Enhancements

Tresna Maulana Fahrudin ¹, Nobuo Funabiki ^{1,*}, Komang Candra Brata ^{1,2}, Inzali Naing ¹,
Soe Thandar Aung ¹, Amri Muhaimin ³ and Dwi Arman Prasetya ³

¹ Department of Information and Communication Systems, Okayama University, Okayama 700-8530, Japan; tresnamf@s.okayama-u.ac.jp (T.M.F.); k.candra.brata@ub.ac.id (K.C.B.); psdy1zwf@s.okayama-u.ac.jp (I.N.); soethandar@s.okayama-u.ac.jp (S.T.A.)

² Department of Informatics Engineering, Universitas Brawijaya, Malang 65145, Indonesia

³ Department of Data Science, Universitas Pembangunan Nasional Veteran Jawa Timur, Surabaya 60294, Indonesia; amri.muhaimin.stat@upnjatim.ac.id (A.M.); arman.prasetya.sada@upnjatim.ac.id (D.A.P.)

* Correspondence: funabiki@okayama-u.ac.jp

Abstract: Nowadays, accessibility to academic papers has been significantly improved with electric publications on the internet, where *open access* has become common. At the same time, it has increased workloads in literature surveys for researchers who usually manually download PDF files and check their contents. To solve this drawback, we have proposed a *reference paper collection system* using a *web scraping* technology and natural language models. However, our previous system often finds a limited number of relevant reference papers after taking long time, since it relies on one paper search website and runs on a single thread at a multi-core CPU. In this paper, we present an improved *reference paper collection system* with three enhancements to solve them: (1) integrating the APIs from multiple paper search web sites, namely, the *bulk search endpoint* in the *Semantic Scholar* API, the *article search endpoint* in the *DOAJ* API, and the *search and fetch endpoint* in the *PubMed* API to retrieve article metadata, (2) running the program on multiple threads for multi-core CPU, and (3) implementing *Dynamic URL Redirection*, *Regex-based URL Parsing*, and *HTML Scraping with URL Extraction* for fast checking of PDF file accessibility, along with sentence embedding to assess relevance based on semantic similarity. For evaluations, we compare the number of obtained reference papers and the response time between the proposal, our previous work, and common literature search tools in five reference paper queries. The results show that the proposal increases the number of relevant reference papers by 64.38% and reduces the time by 59.78% on average compared to our previous work, while outperforming common literature search tools in reference papers. Thus, the effectiveness of the proposed system has been demonstrated in our experiments.

Keywords: reference paper collection; multiple API integration; PDF accessibility; open access; multiple threads



Academic Editor: Paolo Bellavista

Received: 8 March 2025

Revised: 24 April 2025

Accepted: 25 April 2025

Published: 28 April 2025

Citation: Fahrudin, T.M.; Funabiki, N.; Brata, K.C.; Naing, I.; Aung, S.T.; Muhaimin, A.; Prasetya, D.A. An Improved Reference Paper Collection System Using Web Scrapping with Three Enhancements. *Future Internet* **2025**, *17*, 195. <https://doi.org/10.3390/fi17050195>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Currently, the accessibility of academic papers has significantly improved with the spread of electronic publications on the internet. In particular, *open-access* publications have become common due to advantages in publication costs, distribution costs, and visibility, compared to *closed access* ones [1]. Because of the limitations on accessing papers, *closed access* publications may lead to illegal actions, such as the use of *Sci-Hub* [2], paywall bypassing [3], unauthorized credential sharing [4], and terms-violating scraping [5]. On

the other hand, *open-access* publications offer ethical solutions for researchers to access needed articles with low costs. Platforms like *Google Scholar*, *Publish or Perish*, *OpenAlex*, and *OpenAIRE* have been widely recognized as search tools that facilitate access to academic articles [6].

Previously, we have proposed a *reference paper collection system* using a *web scraping technology* and AI models for natural languages. This system collects the site information for reference papers through *Google Scholar* by running a script program to control the search process with *Selenium*. *Selenium* is a popular open source web scraping software. The system provides PDF files of related papers by downloading them from the collected websites and analysing the similarity of their contents to the user request using open source natural language models [7].

However, this system can often find a limited number of relevant reference papers, even after taking long time, since it relies on a single paper search site and runs on a single thread at a multi-core CPU. The system retrieves around 25 to 50 papers per request, with an average relevance accuracy of 53.29% and a response time ranging from 41 to 507 s. The reliance on a single search site may limit the diversity of references obtained and can trigger suspicion because repetitive requests are sent without a time delay. Furthermore, it can increase the risk of violating the terms of service and prohibiting automated scraping, which can lead to *Internet protocol blocking*, and make the site inaccessible and the system unusable.

To overcome these limitations, in this paper, we present an improved *reference paper collection system* with three enhancements. For its efficient implementation and program management, we newly designed and implemented the whole system from scratch using proper frameworks and libraries.

The first enhancement is the integration of the APIs from three paper search sites. They are the *bulk search endpoint* in the *Semantic Scholar* API, the *article search endpoint* in the *DOAJ* API, and the *search and fetch endpoint* in the *PubMed* API to retrieve article metadata. *Semantic Scholar* covers computer science, geoscience, neuroscience, and others [8]. *DOAJ* covers a broader scope [9]. *PubMed* focuses on biotechnology, medical, and health domains [10].

The second enhancement is the implementation of the program for multi-threading to run the multiple paper searches simultaneously on a multi-core CPU [11,12]. It can reduce the response time drastically.

The third enhancement is the implementations of *Dynamic URL Redirection*, *Regex-based URL Parsing*, and *HTML Scraping with URL Extraction*. The extracting sentence embedding and their semantic information using *Sentence Transformers models (SBERT)*, along with text similarity score using *Cosine Similarity* is adopted. This approach can offer fast checking of PDF file accessibility and improve accuracy relevance.

To increase availability and usability, this system is implemented as a web application system. On the client-side, *Streamlit* is adopted to rapidly build interactive web apps, and display the user interface [13]. On the server-side, the main program has been implemented using *Python* and supporting libraries. By utilizing *endpoints* from external web APIs [14], it retrieves article metadata in a *JSON* format using the *requests* library [15] and in an *XML* format using *ElementTree* library [16,17]. The program employs the *concurrent.futures* library to run multiple search site accesses on multi-threading and asynchronous execution to enhance processing speed by handling multiple requests concurrently [18]. The program employs *BeautifulSoup* and *Regular Expression* libraries [19] to parse HTML contents from article URLs and check accessible PDF files. Furthermore, the program employs *PyPDF2* for extracting keyword from PDF files [20], and *sentence-transformers* library for extracting semantic representation of a sentence and check their similarity [21].

For evaluations of the proposal, we implemented the prototype environment, and selected five reference paper queries covering different topics. Then, we compared the

number of available relevant references with full-text PDF access by the proposal with our previous system [22], and examined the impact of multi-threading on the response time [23]. The proposal increases the number of relevant references with full-text PDF access by 64.38% on average, and reduces the response time by 59.78% on average.

After them, we compared the number of references by the proposal with common literature search tools such as *Publish or Perish* [24], *OpenAIRE* [25], *OpenAlex* [26], and *Google Scholar* [27]. The results show that the proposal outperforms them in providing the number of relevant references and full-text PDF access. Finally, we discussed the proper external web APIs source for reference papers based on the perspective of the proposed system and the limitations.

The paper is structured as follows: Section 2 introduces related works in the literature. Section 3 presents the adopted tools. Section 4 presents three enhancements of the *reference paper collection system*. Section 5 presents multiple web API integration. Section 6 presents multi-thread implementation. Section 7 presents fast checking of PDF file accessibility. Section 8 presents keyword matching and text similarity. Section 9 evaluates the performance of the proposal. Section 10 concludes this paper with brief future works.

2. Related Works in the Literature

In this section, we discuss works related to this research.

2.1. Multi-Threading for Multiple Search Sites in Academic Paper Retrieval

First, we discuss papers on implementing multi-threading or parallelism techniques for running multiple search sites for academic papers. Multi-threading enables concurrent executions of multiple tasks related to requesting data from multiple sources. It can maximize the throughput for collecting papers and reduce the response time.

In [28], Jian et al. presented the *CiteSeerX* milestone as an *open-access*-based scholarly Big Data service. *CiteSeerX* collected data using an incremental web crawler developed with *Django*. Metadata processing used *Perl* with a *multithreaded Java wrapper* that was responsible for retrieving documents from the crawl database and managing parallel tasks [29]. They reported that almost 40–50% of the total PDF documents successfully found per day and identified as scholarly are stored in the database.

In [30], Petr et al. introduced *CORE*, a service that provides *open-access* to research publications through bulk data access and downloads them via *RESTful* APIs. The system ensures up-to-date data with the *FastSync* mechanism and manages all the running tasks through *The CORE Harvesting System (CHARS)*. However, this paper highlights the limitation of determining the optimal number of workers, as allocation relies on empirical methods.

In [31], Jason et al. introduced *OpenAlex*, an open bibliographic catalogue covering a wide range of scientific papers and research information. *OpenAlex* gathers heterogeneous directed graph datasets which collected data from *Microsoft Academic Graph*, *Crossref*, *PubMed*, institutional repositories, and e-prints. *OpenAlex* used *async* in *JavaScript* to handle asynchronous operations and *parallel-transform* to process data streams in parallel [32].

In [33], Patricia et al. proposed *PaperBot*, an open source web-based software for automating the search process and indexing biomedical literature. It retrieved metadata and PDFs using multiple database APIs. Using *MongoDB*, it achieves scalable cluster operations to enable parallel processing and reducing latency. The results showed the increase in the throughput by five times in paper collections. However, storing PDF files as blobs led to increasing the RAM usage.

In [34], Claudio et al. proposed *OpenAIRE* workflows for data managements to support open science e-infrastructure, allowing shares and reuses of research products. *OpenAIRE*

utilized a graph-based *Information Space Graph (ISG)* to collect heterogeneous data from semantically linked research products, while the *D-Net Framework* orchestrated workflows. *ISG* used *Apache HBase* for scalable columnar storage and *MapReduce* for parallel processing of large graph datasets.

In [35], Marcin et al. introduced *Paperity Central*, a global universal catalogue combining gold and green *open-access* scholarly literature. *Paperity Central* used an automatic aggregator to accelerate collections of large-scale new publications from *open-access* sources and repositories while connecting various data items based on semantics. However, this paper has not discussed how the aggregation mechanism distributes tasks to workers while retrieving metadata from various data sources.

Various academic search engines have been implemented for different types of multi-threading or parallel programming. Notably, it emphasizes the selection of multi-threading libraries and adjusts the number of workers based on the complexity and the system environment to maximize the throughput. Since the proposed system integrates multiple search sites and runs concurrently, implementing multi-threading is highly recommended.

2.2. Web Scraping Methods for Data Extraction

Second, we review studies on implementing web scraping methods for data extraction. They use techniques of *URL redirect utility*, *Regular Expressions*, and *BeautifulSoup* for automating and customizing web page information extraction.

In [36], Dande et al. proposed *Web URL Redirect Utility* to validate URL links and generate reports. This utility detects URL bottlenecks and enhances the performance of each web page by dynamic URL redirection that runs in a parallel programming approach. The execution efficiency of the developed system reaches 100%, with the execution time reduced by 98–99%. Inspired by this approach, we redirect inaccessible PDF URLs to an alternative URL path with a dynamic article ID to improve full-text access.

In [37], Jian-Ping et al. proposed using a regular expression to extract weather data from *Sohu*, *Sina*, and *Tencent* websites. The platform utilized *java.util.regex* package in Java, which provides classes for parsing pages with regular expressions. Leveraging this method, we identify the URL path structure on a journal article page built using *Open Journal Systems (OJS)* and extract PDF ID based on regular expression pattern matching.

In [38], Sakshi et al. proposed an exploration of *BeautifulSoup* as a Python library for web scraping. Their practical studies demonstrated efficient and versatile use of *BeautifulSoup* for news data mining, academic research data collections, and product information extractions. We extracted PDF URLs from scientific journal article pages, which were redirected from *Digital Object Identifier (DOI)* URL.

2.3. Sentence Transformer Models for Semantic Sentence Embedding

Third, we review studies on implementing *Sentence Transformers models* for semantic sentence embedding. They used several variants of models, such as *MiniLM L6*, *MiniLM L12*, *MPNet*, and *DistilRoBERTa* to obtain sentence representations that understand contextual meaning rather than focus on individual words.

In [39], Galli et al. proposed comparing *pre-trained sentence transformer* models to accelerate the systematic review process in the medical field of peri-implantitis. In their study, they reported that *MiniLM L6* and *L12* are computationally efficient but compromise the accuracy of the generated semantic information. On the other hand, *MPNet* and *DistilRoBERTa* offer better semantic information accuracy but are significantly more time-consuming.

In line with the study by Colangelo et al. [40], which states that there is a trade-off between accuracy and computational efficiency based on the underlying *pre-trained* model architecture, we adopt *MiniLM L6* to assess semantic similarity between keyword extraction

from PDF and query without compromising processing speed. Given that the *reference paper collection system* requires fast response time while maintaining relevance accuracy.

2.4. Performance Measurement for Academic Paper Retrieval

Finally, we review papers on metrics to measure the effectiveness in providing *open-access* articles with downloadable full texts. They include the total number of *open-access* full-text PDFs. Other studies also measure the relevance of the retrieved articles to the input query to assess the quality of the search results.

In [41], Vivek et al. used these metrics to measure the effectiveness of *Google Scholar* in providing free downloadable full-text versions of scientific articles for researchers in regions with limited access to scientific literature.

In [42], Gupta et al. used precision to measure relevance by comparing the number of reference papers that are truly relevant to a user query with the total number of reference papers retrieved by the system.

In [43], Hyunjung recommended to use the response time to measure the search speed, which is defined as the duration from when a query is sent to the search system until the user receives the search results.

3. Adopted Tools

In this section, we describe the software tools adopted in the proposed study.

3.1. External Web APIs

In this paper, we adopt three external web APIs to search and collect reference papers to enhance the efficiency of retrieving bibliographic metadata. They include *Semantic Scholar* API, *DOAJ* API, and *PubMed* API.

3.1.1. Semantic Scholar API

Semantic Scholar provides external web API access through the *Academic Graph API* endpoint using a custom-trained ranker to perform keyword searches. They include *Paper Bulk Search endpoint* and the *Paper Relevance Search endpoint*. We utilize *Paper Bulk Search endpoint* for large-scale query scenarios which can retrieve up to 10,000,000 papers with less resource usage, though it provides less detailed metadata [44].

3.1.2. DOAJ API

DOAJ provides bibliographic metadata for *open-access* journals that support various endpoints, including *CRUD APIs for Applications, Articles, and Journals*, as well as *Bulk and Search APIs*. We utilize *Search API endpoint* that enables keyword searches and filtering data based on certain parameters, or even performs complex queries to get specific results [45].

3.1.3. PubMed API

PubMed has its server program developed by the *National Center for Biotechnology Information (NCBI)*, called *Entrez Programming Utilities (E-utilities)*. *E-utilities* can be accessed via URL with specific parameters to search and obtain data from various *NCBI* sources. *NCBI E-utilities* provides endpoints, including *EInfo*, *ESearch*, *EPost*, *ESummary*, *EFetch*, and several others. We utilize *ESearch endpoint* to obtain a list of *UIDs* or unique identifiers for each article in the *PubMed* database that matches the user's text query, while *EFetch endpoint* is used to retrieve the details for each input list of *UIDs* [46].

3.2. JSON and ElementTree Library

To facilitate data retrieval from external web APIs, we adjust the data structure according to each platform's format. *Semantic Scholar* and *DOAJ* APIs return in *JSON* data format,

while *PubMed* APIs return in XML, inherently hierarchical. *Python* provides the *requests* library to request a specific URL. It will return a response object to access the content such as bibliographic metadata. It can load JSON data into a *Python* object using *response.json()* [47] module. It can parse XML data using the *ElementTree* [48] module.

3.3. BeautifulSoup Library

BeautifulSoup is a *Python* library used to retrieve HTML and XML data. *BeautifulSoup* functions as a parser to separate HTML components into a series of elements that are easy to read. This library is useful in *web scraping* to collect data from websites. There are many functions available in *BeautifulSoup*, such as *soup.find()*, *soup.find_all()*, *soup.get_text()*, *soup.prettify()* and others. We utilize *BeautifulSoup('html.parser')* and *soup.find_all()* to parse HTML content and extract relevant elements [49].

3.4. Regular Expression Library

Regular Expression, abbreviated as *RegEx*, is a sequence of characters that form a search pattern used to matching with a comparison string. *Python* provides *re*, a built-in package to work with *Regular Expression*. The *re* module has several functions, such as *re.match()*, *re.search()*, and *re.findall()*. The *re* module also provides characters used to form patterns, such as metacharacters, special sequences, and sets of characters which has its own distinct meaning. We utilize *re.search()* to find a pattern anywhere in the string [50].

3.5. Concurrent Futures Library

Concurrent.futures has several classes and subclasses in *Python*, including *ThreadPoolExecutor*, *executor.submit*, *as_completed*, and *future.result()*. *ThreadPoolExecutor* is used to run multiple tasks simultaneously with a pool of threads. *executor.submit* is used to submit a task to the thread pool to be executed asynchronously. Additionally, *as_completed* is a generator function used to iterate over *future objects* after completion and is useful for processing the results of a task immediately after completion without waiting for all tasks to complete. Finally, *future.result()* is used to get the result of a task execution [51].

3.6. PyPDF2 Library

PyPDF2 is an open-source *Python* library that can split, trim, merge, and convert PDF files. This library also works well to extract metadata from PDF files. *PyPDF2* is quite lightweight and does not require large external dependencies compared to other PDF extractor libraries. *PdfReader* class from *PyPDF2* helps load and parse the structure of a PDF document. We utilize *PyPDF2* to extract keywords from PDF files that the system has successfully found [52].

3.7. Sentence Transformers Library

Sentence Transformers is a *PyTorch* and *TensorFlow*-based library built on *Hugging Face's transformer* architecture to transform a document's text into embedding vectors in *Natural Language Processing* tasks. *Sentence Transformers* provides various *pre-trained SBERT* models and their related architecture. We utilize *MiniLM L6 v1*, trained on 1 billion sentence pairs, to understand semantic similarity. This model is equipped with 6 encoder layers, 384 hidden dimensions, 128 tokens as the maximum sequence length, and offers efficiency in computational speed and good semantic search performance. We also utilize *util.cos_sim()* module from *sentence_transformers* library to measure the semantic closeness between papers with input queries based on *Cosine Similarity* [53].

3.8. Streamlit Library

Streamlit is an open source *Python* framework used to develop interactive websites quickly and easily, and display data dynamically with few line codes. *Streamlit* is popular for data scientists, data analysts, and AI/ML engineers to create data-driven applications without requiring in-depth web development skills. *Streamlit* can be integrated with other *Python* libraries such as *Pandas*, *NumPy*, and *Matplotlib*, especially for data analysis and visualization [54].

4. Design of Improved Reference Paper Collection System

In this section, we present the overview of the improved *reference paper collection system* using web scraping with three enhancements.

4.1. System Overview

This web-based application system consists of the client-side and the server-side. For the client-side, *Streamlit* [54] is used for connections with the server-side and user interfaces. For the server-side, *Python* with several libraries is used. *JSON requests* and *ElementTree* are adopted for retrieving data, *concurrent futures* is for multi-threading, *BeautifulSoup* and *Regular Expression* are used for web scraping, *PyPDF2* for extracting keywords from PDF files, and *Sentence Transformers* is for extracting semantic sentence embedding and checking the similarities. Figure 1 illustrates the overview of a *reference paper collection system* in the proposal.

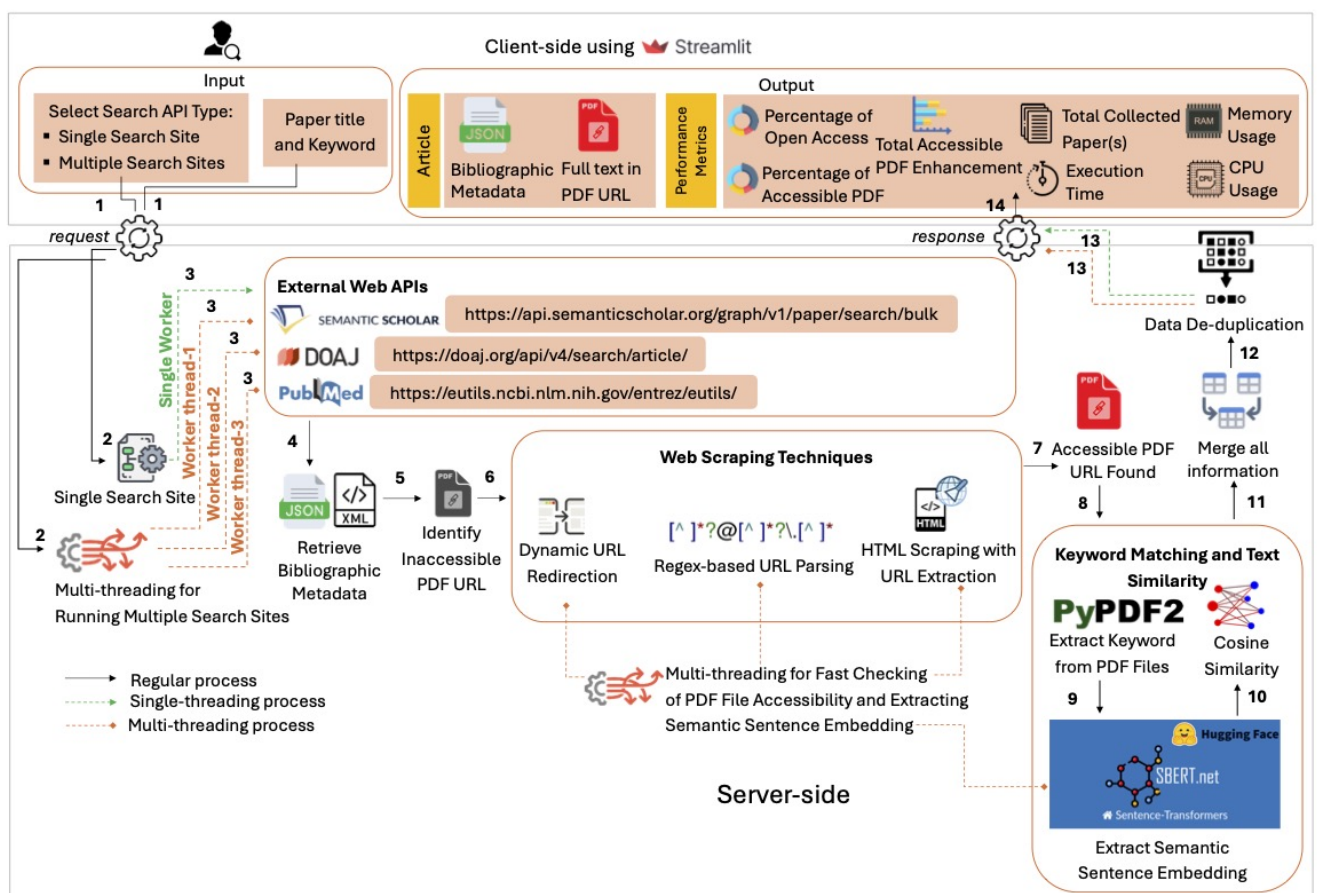


Figure 1. Overview of improved *reference paper collection system*.

4.1.1. Input

The system receives input from the user through a client-side interface developed using *Streamlit*. This input consists of search API types from the radio buttons, paper title and keyword search queries from the text forms. Once the user submits a request, the data is forwarded to the server-side for further processing.

4.1.2. Process

On the server-side, the system processes the input, and will choose either a single or multiple search site to make request external web APIs. External web APIs receive the data requests and respond by providing bibliographic metadata in the *JSON* or *XML* format. The system retrieves the metadata and identifies the inaccessible PDF URLs, where *web scraping* is implemented to check the accessibility of PDFs running in multi-threading. Once an accessible PDF URL is found, the system encodes the paper title and extracted keywords from PDF files, as well as the title and keyword queries by user, into semantic sentence embeddings to calculate similarity. Subsequently, the system merges all the retrieved information, removes the duplicate metadata records, and prepares the result in a structured format before being sent back to the client-side.

4.1.3. Output

Finally, the client-side displays the paper list by sorting them based on the highest similarity scores to the requested keyword search queries, along with performance metrics information.

4.2. Client-Side Design

On the client-side, a user begins its use by selecting the search API type and filling the paper title and keyword search queries at the input forms in the user interface. Either the single search site or the multiple search site should be selected. The *single search site* retrieves data from one of *Semantic Scholar* API, *DOAJ* API, and *PubMed* API. The *multiple search site* retrieves data from the three APIs that run in multi-threading.

After the user completes the inputs, the user presses the enter key and waits for a response from the server-side. When the server-side successfully retrieves the data and sends it to the client-side, the client-side will display the paper list of article metadata, performance metrics, and export them into a CSV file.

Users can access the application through a browser at *localhost:8501*, which is the default port used by *Streamlit*. Figure 2 shows the user interface of the proposed system. It displays (a) the main page, and (b) performance metrics in graphical presentations.

In the main page, a user is allowed to explore the search result in the table including the searching and sorting paper list. This table includes the paper ID, the paper title, the abstract, the publication year, the authors, the citation, the discipline or field of science, the document type, the journal name, the volume, the page range, the open access category, the PDF URL, the similarity score, and the related information in each reference paper.

In the performance metrics page, a user receives the information including (1) the percentage of *open-access* and accessible PDF files, (2) the number of papers with downloadable PDFs, (3) the total number of collected papers, (4) the resource usage performance such as the response time, the memory usage, and the CPU usage, and (5) the CSV file for exporting the collected paper list.

岡山大学 Welcome to ARPACS Project
A Reference Paper Collection System - Open Access Local Search API

Open Access Paper Retrieval

Choose the API:
☐ Semantic Scholar API
☐ DOAJ API
☐ PubMed API
☒ Multiple API Integration

Enter your query:
Healthcare Chatbot Natural Language Processing

Enter up to 10 keywords for refining search:
 Enter keywords:
 Chatbot Conversational Agent Natural Language Processing Health Support Question Answering System Press enter to add more

Search

Searching for "Healthcare Chatbot Natural Language Processing" with keywords: ["Chatbot", "Conversational Agent", "Natural Language Processing", "Health Support", "Question Answering System"]

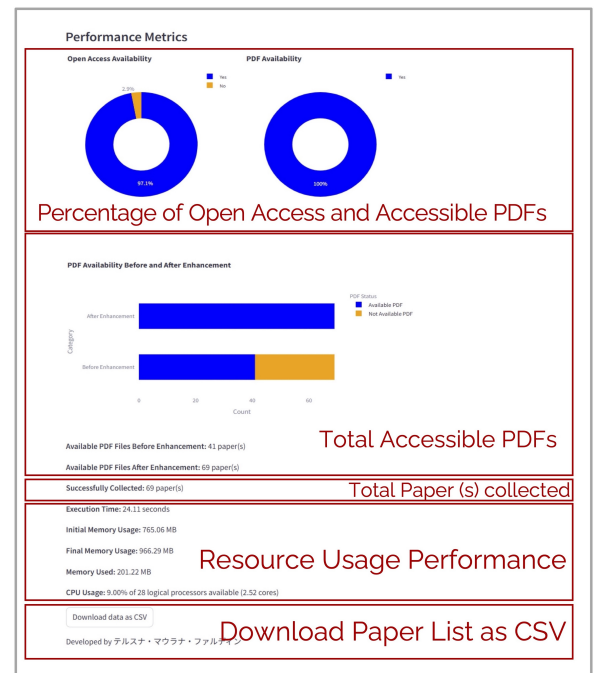
Fetching data from multiple APIs...

Data fetched in 24.11 seconds!

	Open Access PDF URL	PDF Av.	URL	DOI	Source	Extrich	Similarity Score
14	https://www.fim.com/papers/2023/6/0304.pdf	Yes	https://	10.369/	Seman	Artific	0.7527
1	https://www.mdpi.com/2099-5903/10/7/2131/pdf	Yes	https://	10.339/	Seman	Artific	0.699
118	https://nature.com/articles/141746-622-05569-6-g	Yes	https://	10.1038/	DOAJ	-	0.6924
117	https://pmc.ncbi.nlm.nih.gov/articles/PMC38130/	Yes	https://	10.1038/	PubMe	large ta	0.6757
151	https://pmc.ncbi.nlm.nih.gov/articles/PMC28546/	Yes	https://	10.100/	PubMe	-	0.672
138	https://pmc.ncbi.nlm.nih.gov/articles/PMC28735/	Yes	https://	10.100/	PubMe	secut	0.6602
210	https://pmc.ncbi.nlm.nih.gov/articles/PMC28735/	Yes	https://	10.100/	PubMe	secut	0.6438
144	https://pmc.ncbi.nlm.nih.gov/articles/PMC38984/	Yes	https://	10.117/	PubMe	Artific	0.6337
22	https://doi.org/10.32620/cell2173172	Yes	https://	10.326/	Seman	Artific	0.6336
110	https://www.ncbi.nlm.nih.gov/pmc/articles/PMC38130/	Yes	https://	10.1038/	PubMe	large ta	0.6757

Page 1 of 1

(a)



(b)

Figure 2. User interface: (a) main page; (b) performance metrics.

4.3. Server-Side Design

On the server side, a *Python* program handles the request by responding to the search API type and the queries given by the user. The web browser sends an *HTTP GET* request to the web server with related parameters. The internal system utilizes multithreading using the *concurrent.futures* library to allow multiple search API requests to be processed in parallel. The *HTTP GET* request is used because it is intended only for fetching data. The web server retrieves the requested data from the file system and returns an *HTTP* response status code of 200 when it successfully retrieves the bibliographic metadata. The web server may return a different status code, such as 404, when the requested data is not found.

The web server, which handles the requests from the client-side and manages the API calls, detects the request and forwards it to the application server according to the respective platform API *endpoint*. The application server identifies the request based on the parameters, including the search API type and the keyword search queries. The application server then gets the required information from the database through the search API *endpoint*. The */search/bulk endpoint* is used to retrieve metadata from *Semantic Scholar* database, the */search/article endpoint* is used in *DOAJ* database, and the */entrez/entrez/ endpoint* is used in *PubMed* database. The application server sends back the metadata obtained to the web server.

After the metadata is retrieved from the web server, the internal system parses and extracts metadata using *response.json()* and *ElementTree*. Since the data sources return responses in different formats, we standardize all extracted data and reformat them into a *JSON* schema to facilitate data integration and ensure metadata consistencies. Subsequently, the metadata is processed in a series of data processing stages and stored in a *JSON* format. Finally, the final metadata is sent to the user as a response from the program. Figure 3 illustrates the server-side design of a *reference paper collection system*.

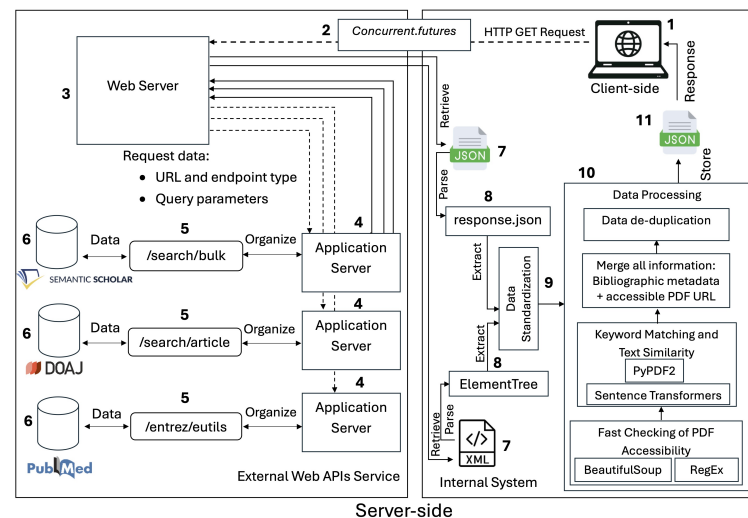


Figure 3. Server-side design.

5. Multiple Web API Integration

In this section, we present the multiple web API integration that leverages the search API type *endpoints* from three platforms, and describes the workflow of the single and multiple search sites.

5.1. Bulk Search Endpoint in Semantic Scholar

Semantic Scholar provides the *bulk search endpoint* with a structure consisting of the base URL: <https://api.semanticscholar.org/graph/v1> and the specific *endpoint's* resource path: *"/paper/search/bulk"* [44]. This *endpoint* retrieves a large amount of bibliographic metadata in a single request. We adopted this *endpoint* and implemented it into a module we developed called *run-semanticsc-api.py*. This module contains a main function and subfunction that organize the request metadata to the *Semantic Scholar* API. Table 1 shows an example of article metadata retrieved from the *bulk search endpoint*.

5.2. Article Search Endpoint in DOAJ

DOAJ provides the *article search endpoint* with a structure consisting of the base URL: <https://doaj.org/api/v4/> and the specific *endpoint's* resource path: *"/search/articles"* [45]. This *endpoint* offers the advantage of returning the bibliographic metadata with more specific results based on complex keyword search queries. We adopted this *endpoint* and implemented it into the module called *run-doaj-api.py*. Table 2 shows an example of the article metadata retrieved from the *article search endpoint*.

5.3. Search and Fetch Endpoint in PubMed

PubMed provides the *search and fetch endpoint* with a structure consisting of the base URL: <https://eutils.ncbi.nlm.nih.gov/entrez/eutils> and two specific *endpoint's* resource paths, including *"/esearch.fcgi"* and *"/efetch.fcgi"* [46]. This *endpoint* offers the advantage of searching for articles based on the search criteria and retrieving more detailed information about the articles found. We adopted the *endpoint* and implemented it into the module called *run-pubmed-api.py*. Table 3 shows an example of the article metadata retrieved from the *search and fetch endpoint*.

Table 1. Article bibliographic metadata from Semantic Scholar API.

Paper ID	Title	Authors	Discipline	Journal	Publication Date	Open-Access	Web Page URL	Article PDF URL	DOI
7541082a44ae16ce1ca60a55ae8b4e37d1332d55	Deep learning automation of radiographic patterns for hallux valgus diagnosis	Angela Hussain, Cadence Lee, Eric Hu, Farid Amirouche	Medicine	World Journal of Orthopedics	18 February 2024	Yes	https://www.semanticscholar.org/paper/7541082a44ae16ce1ca60a55ae8b4e37d1332d55	https://www.semanticscholar.org/reader/7541082a44ae16ce1ca60a55ae8b4e37d1332d55	10.5312/wjo.v15.i2.105

Table 2. Article bibliographic metadata from DOAJ API.

Paper ID	Title	Authors	Discipline	Journal	Publication Date	Open-Access	Web Page URL	Article PDF URL	DOI
1117b3c6e5654a25b8c37d231f7be28a	AI nutrition recommendation using a deep generative model and ChatGPT	Ilias Papastratis, Dimitrios Konstantinidis, Petros Daras, Kosmas Dimitropoulos	Medicine, Science	Scientific Reports	30 June 2024	Yes	https://doi.org/10.1038/s41598-024-65438-x	https://nature.com/articles/s41598-024-65438-x.pdf	10.1038/s41598-024-65438-x

Table 3. Article bibliographic metadata from PubMed API.

Paper ID	Title	Authors	Discipline	Journal	Publication Date	Open-Access	Web Page URL	Article PDF URL	DOI
38451905	Systematic evaluation of common natural language processing techniques to codify clinical notes	Tavabi N, Singh M, Pruneski J, Kiapour AM	Humans, Natural Language Processing Electronic Health Records, Language, Quality Improvement	PloS one	1 January 2024	Yes	https://pubmed.ncbi.nlm.nih.gov/38451905	https://pmc.ncbi.nlm.nih.gov/articles/PMC10919678/pdf	10.1371/journal.pone.0298892

5.4. Single Search Site

A single search site refers to a search that only accesses to one database or API to obtain the results. The search result may be more limited compared to the search that combines multiple sources. The proposed system provides the three platform APIs that can be selected and run alternately by the user. Figure 4 illustrates how the system works when a user selects a single search site.

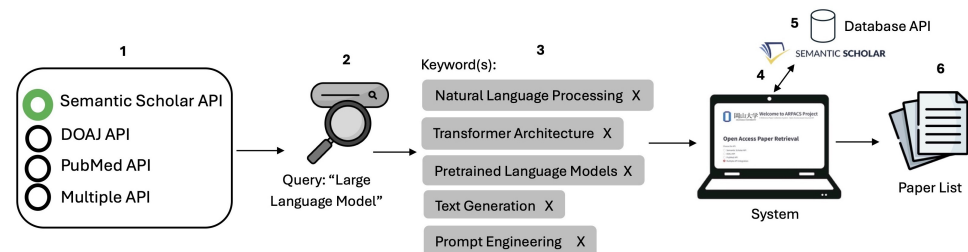


Figure 4. Workflow of single search site.

Figure 4 shows how the system serves the user request with different API options. As an illustration, the system receives input from the user with the search API type “Semantic Scholar API” option, a query “Large Language Model” for the paper title, and several keywords. Subsequently, the system sends them as the parameters to the *Semantic Scholar API endpoint*. *Semantic Scholar* searches for paper titles in its database and sends the results to the system, which then matches both the title and the additional keywords provided by the user as input criteria. After several documents are found, the system performs data processing and displays the paper list to the user.

5.5. Multiple Search Site

The multiple search site refers to a search that combines results from multiple databases or APIs to obtain broader results. Figure 5 illustrates how the system works when the user selects multiple search sites.

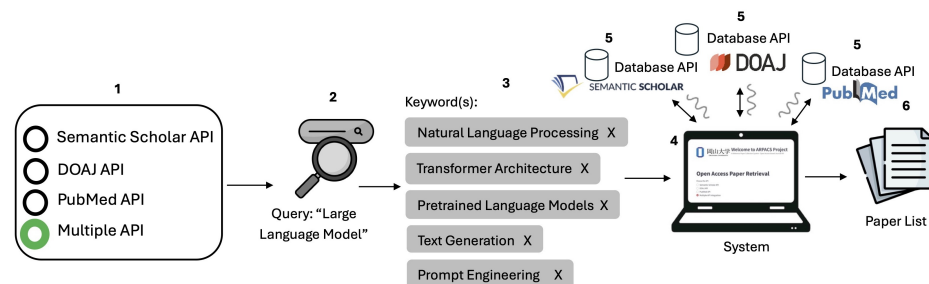


Figure 5. Workflow of multiple search site.

As shown in Figure 5, the system receives the input from a user with the search API type “Multiple API” option, a query “Large Language Model” for paper title, and several keywords. It requests to *Semantic Scholar*, *DOAJ*, and *PubMed endpoint*. Due to requests for multiple APIs, the system involves multi-threads to accelerate retrieving data from each API and running the tasks in parallel. After several documents from multiple APIs are found, the search results from the APIs are standardized in data formats, and the paper list will be displayed to the user.

6. Multi-Thread Implementation

In this section, we present the multi-thread implementation that leverages *ThreadPoolExecutor* in the *Concurrent.futures* library.

Since multiple APIs handle three external API sources simultaneously, sequentially processing these APIs is not feasible, as it would result in an excessively long waiting time for the user. To address this issue, multi-threading is implemented to accelerate processing in parallel.

Figure 6 illustrates the multi-threading implementation using *ThreadPoolExecutor* to run parallel asynchronous processing on multiple search sites. We create *ThreadPoolExecutor* by assigning *max_worker* the desired number of threads [51]. We use *max_worker* of three workers, according to the total external web APIs. After all threads are submitted to the executor, the function in *run-semantic-sc-api.py*, *run-doj-api.py*, and *run-pubmed-api.py* modules send the requests to the respective external web APIs.

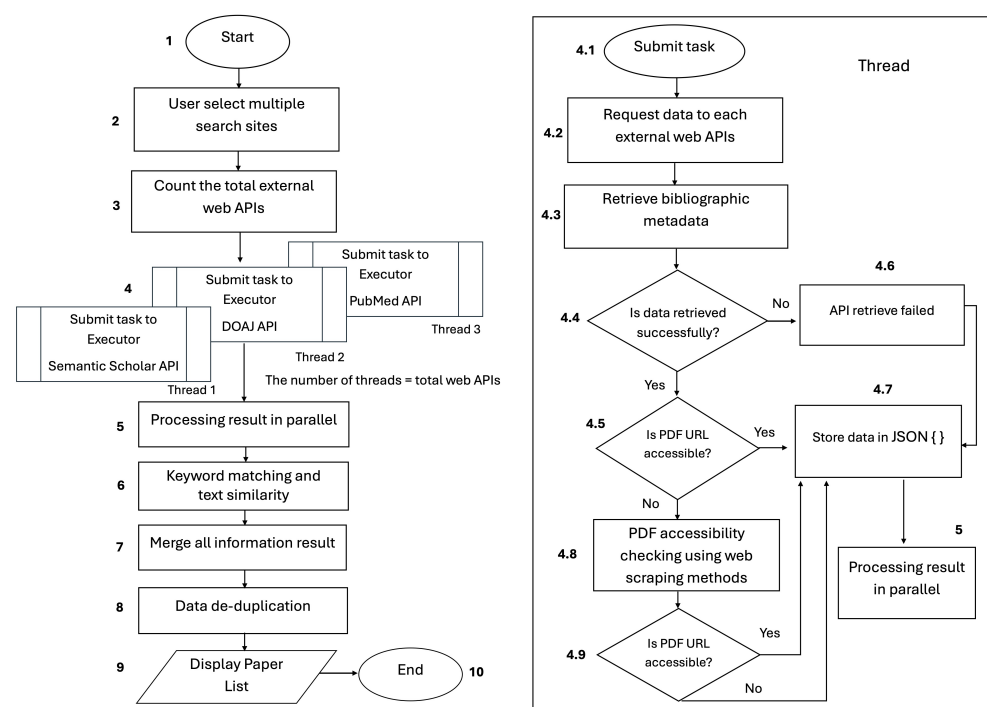


Figure 6. Multi-threading using *ThreadPoolExecutor*.

Once the bibliographic metadata is successfully retrieved, the system will process the PDF checking for the article. On the other hand, if the metadata is unsuccessfully retrieved, it will return the information on web APIs failed. Subsequently, if the PDF URL for each article in the metadata is accessible, the metadata for each article will be stored in a *JSON* format.

If the PDF URL is inaccessible, the system uses *web scraping* technology to check the PDF URL and extract it if found. If the PDF URL remains inaccessible after the web scraping, the article metadata will be stored in a *JSON* format without an accessible PDF URL.

Finally, the system process executes the task results in parallel, performs keyword matching and text similarity, merges all the information obtained into a single metadata record, followed by removing duplicate data and displaying the final paper list to the user.

7. Fast Checking of PDF File Accessibility

In this section, we present the fast checking of the PDF file accessibility to search for accessible PDF URLs in the article metadata obtained using *Dynamic URL Redirection*,

Regex-based URL Parsing, or HTML Scraping with URL Extraction. We also present a flowchart for each method.

7.1. Dynamic URL Redirection

Dynamic URL Redirection was inspired by the research proposed of Dande et al. [36], which involves rerouting URL requests to new destinations, to simplify access to online resources. We apply this method to all the external web APIs.

Figure 7 shows the flowchart of *Dynamic URL Redirection* on *Semantic Scholar*. It begins by sending an HTTP GET request to the *Semantic API endpoint*. The response from the API is received in the *JSON* format, and then, the article metadata is parsed. It is necessary to use a *ThreadPoolExecutor*, and submit the PDF URL of each article and a PDF checking accessibility function to the executor.

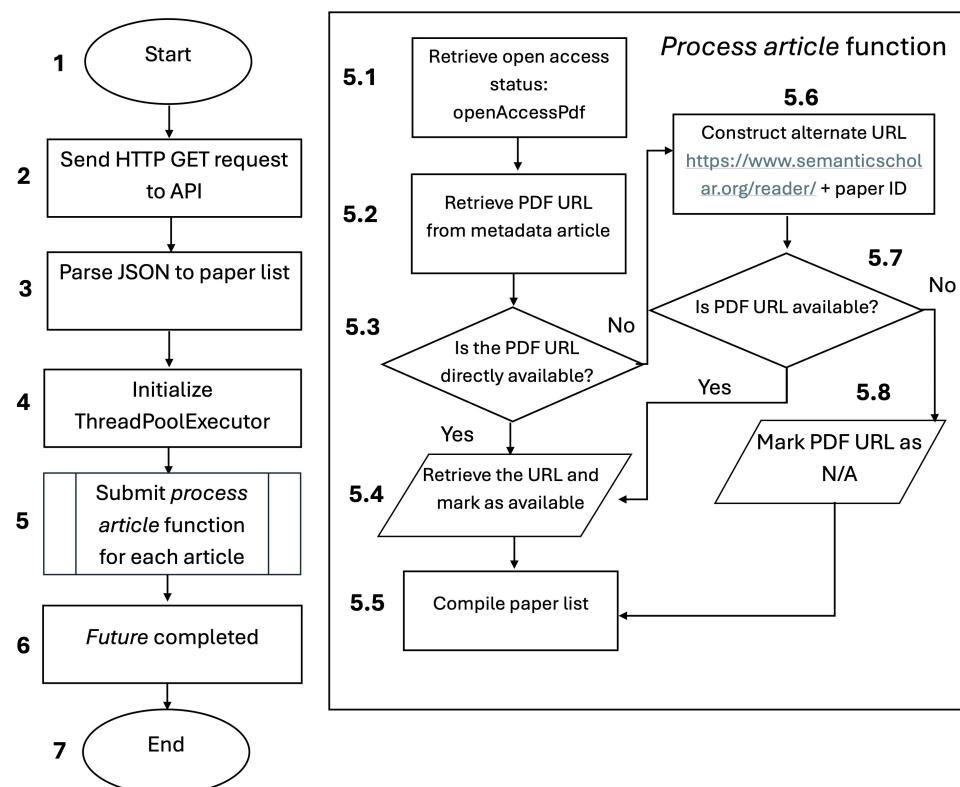


Figure 7. Checking PDF accessibility using *Dynamic URL Redirection* on *Semantic Scholar*.

The function inside retrieves the *open access* status and check the accessibility of the PDF URL in its variable one. If the PDF URL is accessible, the system will retrieve it and mark it as available. Subsequently, article metadata will be compiled into a paper list.

On other hand, if the PDF URL is inaccessible upon accessing the initial URL of *Semantic Scholar* article, construct an alternative URL. The initial URL [https://www.semanticscholar.org/paper/\[PaperID\]](https://www.semanticscholar.org/paper/[PaperID]) is redirected to [https://www.semanticscholar.org/reader/\[PaperID\]](https://www.semanticscholar.org/reader/[PaperID]). The paper ID obtained from the initial URL is assigned to the new URL path. After the redirection to the URL path with the segment “/reader/”, the system enhances the retrieval of an accessible PDF. If the PDF URL is still inaccessible, mark the PDF URL as Not Available (“N/A”). All results are compiled into a paper list and sent to the *future* to get the result of a task execution.

Figure 8 shows the flowchart of *Dynamic URL Redirection* on *DOAJ*. It also utilizes *ThreadPoolExecutor* to complete parallel tasks.

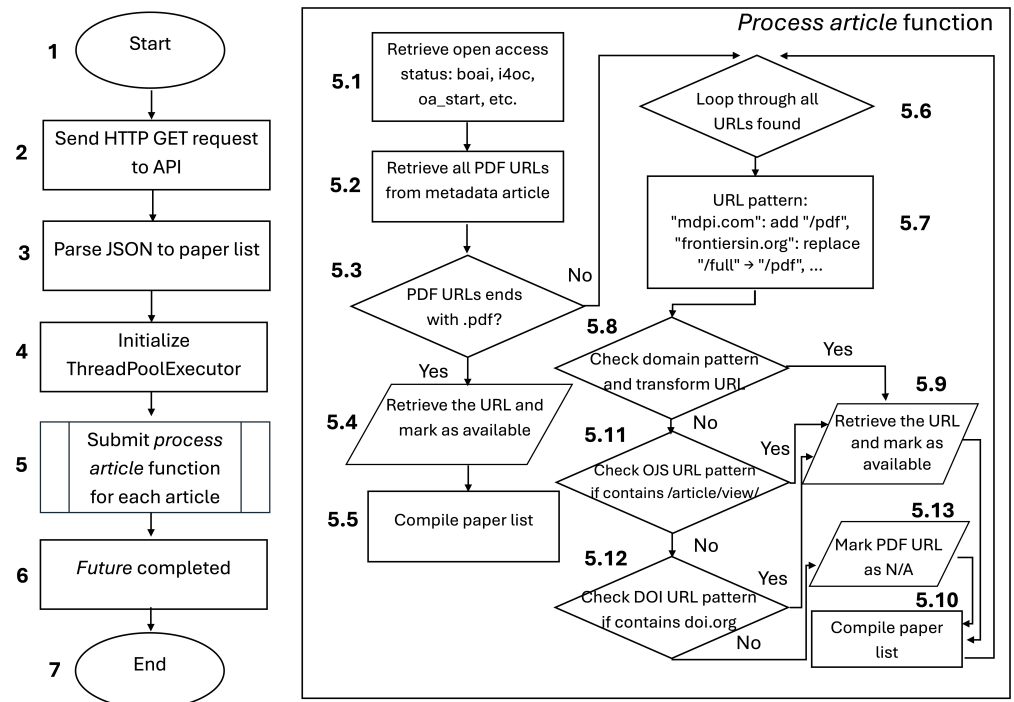


Figure 8. Checking PDF accessibility using *Dynamic URL Redirection* on DOAJ.

The function inside retrieves the *open access* status of article metadata, whether it contains indicators, including the presence of *BOAI compliance*, *I4OC citation openness*, and *OA start date* information. The DOAJ API also provides a list of PDF URLs contained in article metadata. If the PDF URL ends with *.pdf, retrieve the URL and mark it as available. Subsequently, article metadata will be compiled into a paper list.

Since article URLs in the DOAJ database are collected from various journals, adjustments are required using URL pattern replacements when PDF URL is inaccessible. For example, for the *ncbi.nlm.nih.gov* journal URL path, this method replaces “/?tool=EBI” with “/pdf”. On the other hand, the URL paths from several journals, such as *mdpi.com*, *open-research-europe.ec.europa.eu*, *gh.bmj.com*, and *fmch.bmj.com*, only require adding the suffix “/pdf” or “.pdf” to access PDF files directly. If no URL pattern matches, the system will attempt a fallback check using OJS or DOI-specific pattern. If these also fail, the PDF URL is marked as “N/A”.

Figure 9 shows the flowchart diagram of *Dynamic URL Redirection* on PubMed. As with fast checking of PDF file accessibility in *Semantic Scholar* and DOAJ, we also utilize *ThreadPoolExecutor* to complete tasks in parallel on PubMed metadata.

PubMed only provides the limited bibliographic metadata information, while PubMed Central (PMC) provides the complete bibliographic metadata information and the *open-access* full-text PDF. To access the PDF file, we need to retrieve the PMC ID as a value that can be assigned to the PubMed Central URL path. The initial URL from [https://pubmed.ncbi.nlm.nih.gov/\[pmid\]](https://pubmed.ncbi.nlm.nih.gov/[pmid]) is redirected to [https://pmc.ncbi.nlm.nih.gov/articles/\[pmc_id_value\]/pdf](https://pmc.ncbi.nlm.nih.gov/articles/[pmc_id_value]/pdf) and PDF URL is marked as available. If PMC ID is not found, the PDF URL is marked as “N/A”.

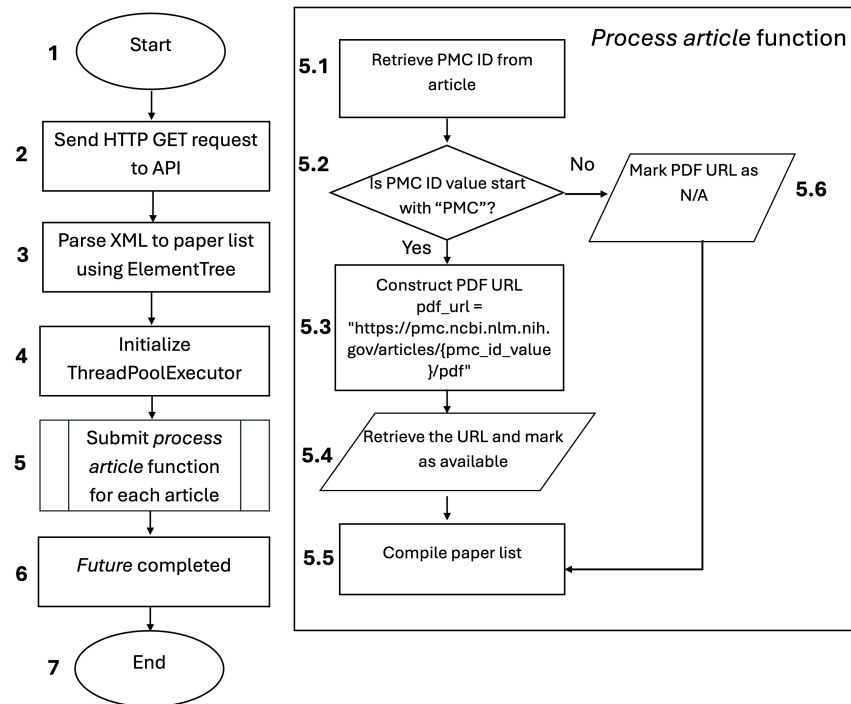


Figure 9. Checking PDF accessibility using *Dynamic URL Redirection* on PubMed.

7.2. Regex-Based URL Parsing

Regex-based URL Parsing [37] is applied to extract the specific URL patterns from *OJS* article URL retrieved from *DOAJ API*. We implement this algorithm to obtain the accessible PDF URL by implementing HTML content parsing using *regular expressions* based on *OJS* URL patterns, as shown in Figure 10.

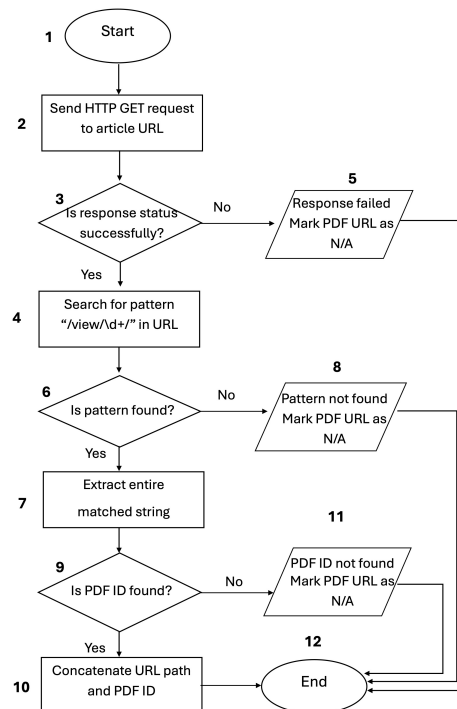


Figure 10. *Regex-based URL parsing* for checking PDF accessibility in *OJS* URL.

This method begins by sending an *HTTP GET* request to fetch the *HTML* content from a URL with a specified timeout parameter. When the request is successful, it searches for a specific URL pattern using *regular expressions* to find the URL path segment “/view/id/”. Once the path segment is found, the process is continued by searching for the PDF ID associated with the article in the *HTML*. If a PDF ID is found, the system returns a concatenation of the base URL path with the PDF ID. Finally, the article PDF URL is accessible. In other conditions, where the request fails to receive a response, the “/view/id/” pattern is not matched, or the PDF ID cannot be found, the PDF URL is marked as “N/A”.

7.3. HTML Scraping with URL Extraction

HTML Scraping with URL Extraction is applied to capture URL from the *HTML* structure of DOI retrieved from DOAJ API. We adopt *web scraping* using the *BeautifulSoup* library [38].

Figure 11 shows the page content that will be parsed using *BeautifulSoup* when the request is successful to facilitate the search for the *HTML* elements. This algorithm searches for all “<a>” elements that have an “href” attribute and checks whether the word “pdf” is included in the URL. If a match is found, the PDF URL will be saved. Then, the algorithm ensures that the PDF URL begins with *http* by adding the domain of the source URL and concatenates them before returning the accessible PDF to the user.

In other conditions, where the request fails to receive a response or “href” attribute containing “pdf” cannot be found, the PDF URL is marked as “N/A”.

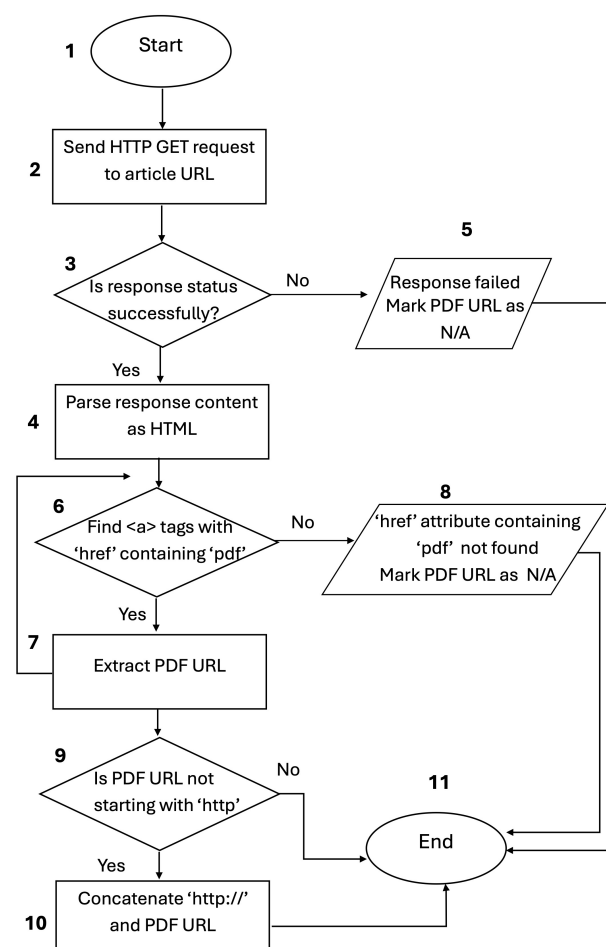


Figure 11. HTML scraping with URL extraction for checking PDF accessibility in DOI URL.

8. Keyword Matching and Text Similarity

In this section, we present the keyword matching and text similarity, including selected document fields such as title, keyword, and abstract. We also present the adoption of semantic sentence embedding models and text similarity using *Cosine Similarity* to calculate the similarity scores of each article and obtain the relevant document. Figure 12 shows the keyword matching and text similarity process using *Sentence Transformers* library.

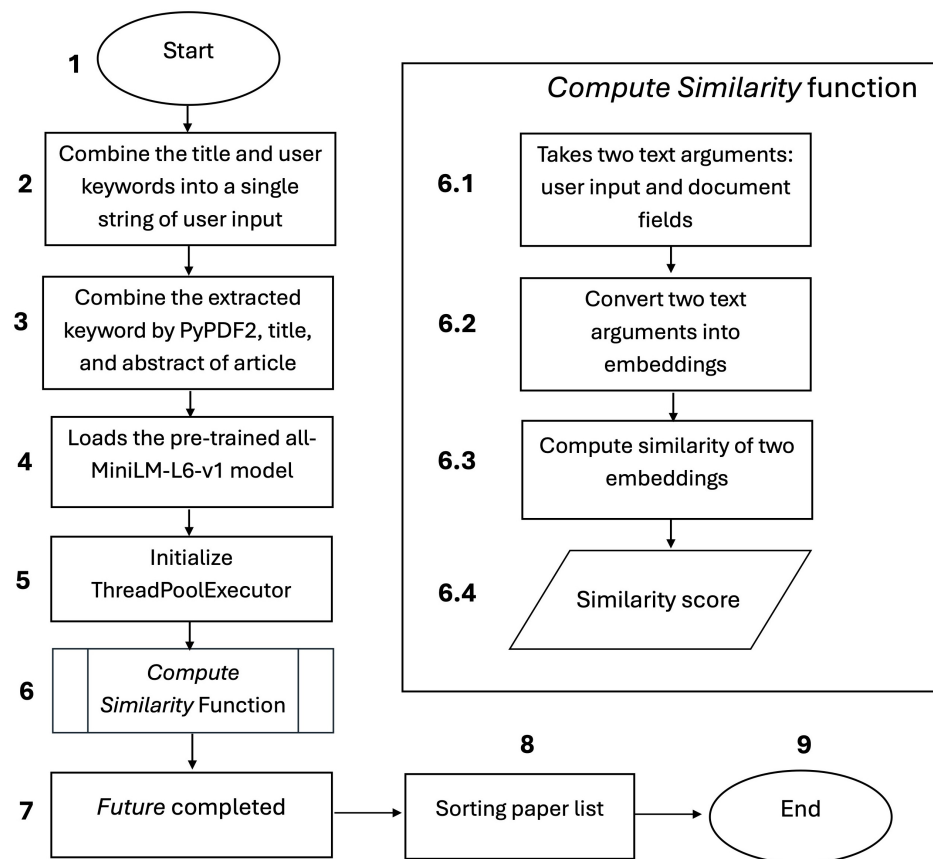


Figure 12. Keyword matching and text similarity using *Sentence Transformers*.

Once the PDF is successfully found, the system will extract keywords from the PDF file using the *PyPDF2* library [52]. The extracted keywords from the PDF will be combined with article metadata such as title and abstract. We utilize the *all-MiniLM-L6-v1* model from *Hugging Face's sentence-transformer* library [53] to extract sentences into vectors to obtain high semantic relationships between the searched papers and user input. This model is fast, lightweight, and suitable for embedding millions of sentences or documents. Despite its small size, its performance is quite competitive for semantic similarity.

After the semantic sentence embedding process, the system computes the similarity score using *Cosine Similarity* between the user input consisting of title and keywords, and the document fields including title, keywords, and abstract. The similarity score ranges from 0 to 1. A similarity score closer to 0 indicates that the two texts are not similar, while a similarity score closer to 1 indicates a high similarity. Finally, the system returns a sorted list of papers in descending order based on similarity scores to the client side.

9. Evaluation and Analysis

In this section, we evaluate and analyse the proposed improved *reference paper collection system*. We discuss testing preparations and performance comparisons with our previous system and common literature search tools. We also discuss the proper web APIs for

reference papers based on the perspective of the proposal. Finally, we discuss the limitation of the proposal.

9.1. Testing Preparations

In preparing this performance evaluation, we used a personal computer equipped with an *Intel(R) Core(TM) i7-14700 (28 CPU) 2.10 GHz* with 20 cores, 28 logical processors, and 16 GB of memory. It runs on the *Windows 11 Pro* operating system. We measured the following performance metrics:

- **PDF Accessibility:** The number of open-access articles with downloadable full texts [41];
- **Relevance:** The number of extracted papers relevant to the query and the user's preference assessment, based on the matching of titles, keywords, research content similarity, and sorted by similarity score [42];
- **Response Time:** The time interval between a user query and the system output [43].

Table 4 presents the five distinct title search queries and keywords to test the system. We determined keyword search queries based on various topics, such as recent trending topics in *Case 5* or a specific topic related to technologies and libraries in *Case 1*. We also used the combination topics of disciplines with scientific domains in *Cases 2, 3, and 4*.

Table 4. Five distinct title and keyword search queries.

Case(s)	Title Search Queries	Keyword (s)
Case 1	Web Scraping using Selenium	Web Scraping, Selenium, Python, Browser Automation, Data Extraction
Case 2	Deep Learning for Skin Detection	Deep learning, Skin Detection, Computer Vision, Convolutional Neural Network, Image Segmentation
Case 3	Healthcare Chatbot Natural Language Processing	Chatbot, Conversational Agent, Natural Language Processing, Health Support, Question Answering System
Case 4	Generative AI for Medical	Generative AI, Synthetic Medical Data, Report Generation, Generative Model, Generative Adversarial Network
Case 5	Large Language Models	Natural Language Processing, Transformer Architecture, Pretrained Language Models, Text Generation, Prompt Engineering

9.2. System Performance Metrics Comparison

We evaluate the performance of the proposal by comparing the number of relevant reference paper with full-text PDF files and the response time, between our previous work and the proposed system, based on title and keyword search queries. Calculating the percentage increase in relevant accessible PDF files obtained and the percentage decrease in the response time are also important to understanding how significantly the proposal enhances the performance.

9.2.1. Relevant Accessible PDFs

Access to full-text PDF articles is essential for researchers to conduct literature reviews. We compare the proposed system with our previous work to evaluate its ability to retrieve relevant accessible PDFs. The proposed system integrates the reference paper sources from multiple APIs, including *Semantic Scholar* [44], *DOAJ* [45], and *PubMed* [46], and collects more accessible PDF files than our previous work relying on *Google Scholar*. This evaluation result is illustrated in Figure 13.

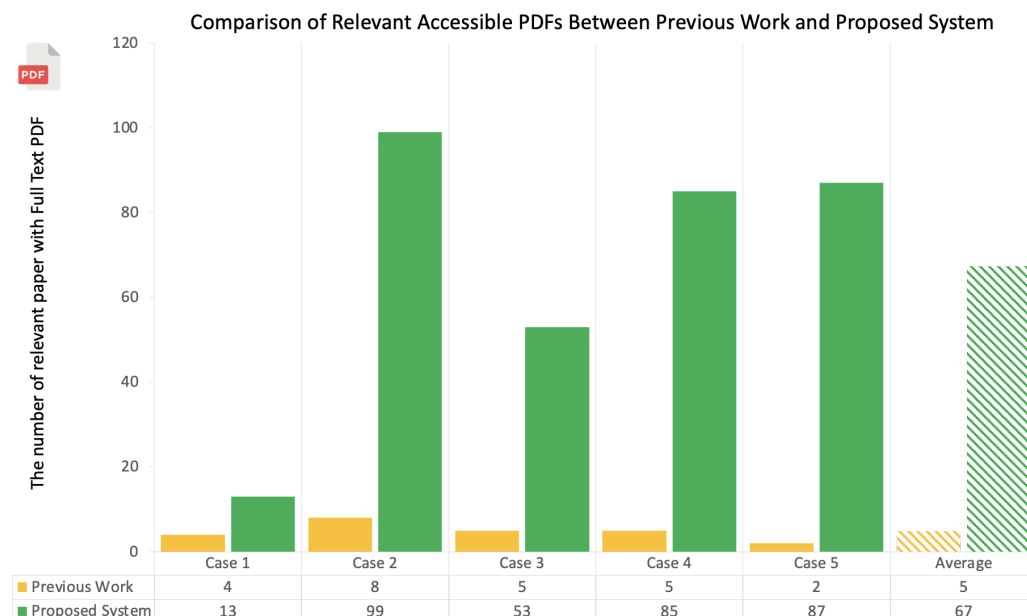


Figure 13. Comparison of relevant accessible PDFs.

Figure 13 shows that the proposed system retrieved an average of 67 relevant accessible PDFs, while our previous work retrieved only 5 PDFs on average for the five distinct title and keyword search queries. Since the proposed system retrieved 87 articles in *Case 5*, it suggests that publications with the topic “Large Language Models” in recent years are well covered in academic literature. The proposed system also demonstrated the ability to retrieve articles with keywords containing disciplines and scientific domains in *Case 2* and *Case 4*, where 99 articles related to “Deep Learning for Skin Detection” and 85 articles related to “Generative AI for Medical” were retrieved, respectively. It indicates that applications of AI in the medical field are growing, with an increasing number of articles being *open-access*.

Unfortunately, the proposed system has a few relevant accessible PDFs in *Case 1* and *Case 3*, where the system only obtained 13 articles related to “Web Scraping using Selenium” and 53 articles related to “Healthcare Chatbot Natural Language Processing”, respectively. This suggests that these specific fields may face limited accesses to the academic literature on *web scraping* that discusses technical aspects using *Selenium*. The applications of *Chatbots* for healthcare may be newer.

Furthermore, to measure the effectiveness of the relevant accessible PDFs enhancement, we calculated the percentage increase. This objective is to calculate how large the percentage of enhancements in relevant accessible PDFs between our previous work and the proposed system, following the *Min–Max normalization*, to obtain a scale of 0–100% [55] in Equations (1) and (2):

$$x_{\text{norm}} = \frac{x - \min(x)}{\max(x) - \min(x)} \quad (1)$$

$$\Delta_{\text{enhance}} = x_{\text{norm2}} - x_{\text{norm1}} \quad (2)$$

We assumed that x represents the number of relevant accessible PDFs for either our previous work or the proposed system, $\min(x)$ does the minimum value as the lowest scale equivalent to 0, and $\max(x)$ does the overall total collected paper. Subsequently, Δ_{enhance} is used to calculate the percentage increase between the normalized values x_{norm2} for the proposed system, and x_{norm1} is for our previous work. The proposal increases the number of available relevant reference papers with full text PDFs by 64.38% on average, compared to previous work. It indicates a significant performance achievement of the

proposal using multiple APIs, *web scraping* methods, and *sentence transformers* in retrieving relevant accessible PDFs.

9.2.2. Response Time

The response time of a system is a critical metric because users expect timely feedback to their queries. We compared the proposed system with our previous work in the response time. Although utilizing multiple APIs, the proposed system demonstrates its efficiency by implementing multi-threading using *ThreadPoolExecutor* [51] for retrieving articles simultaneously, compared to our previous work. The evaluation of the system performance in response time is illustrated in Figure 14.

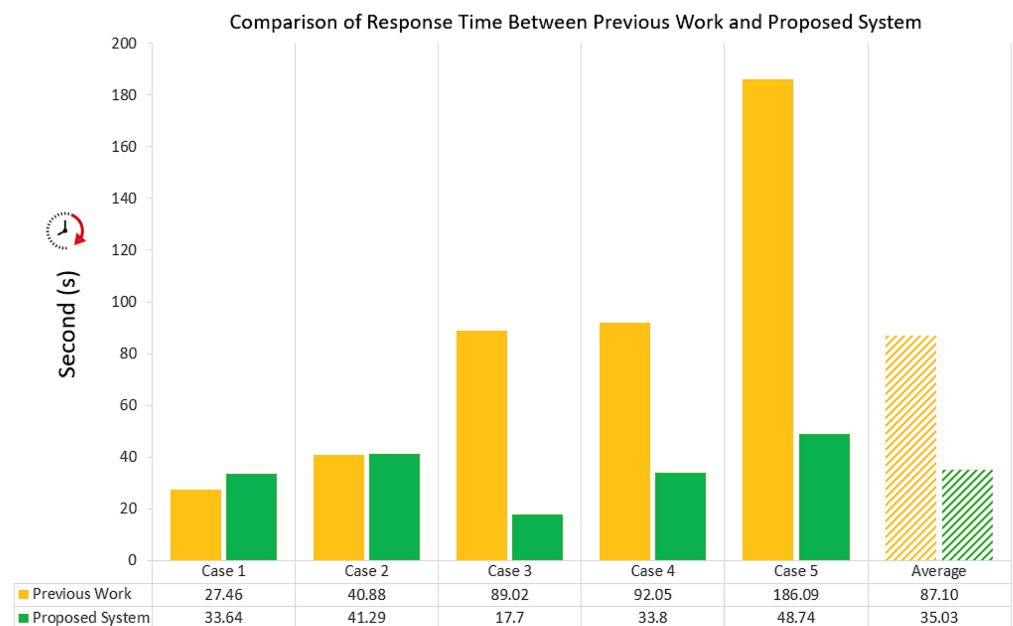


Figure 14. Comparison of response time.

Figure 14 shows that the response time of the proposed system takes 35.03 s on average, while our previous work takes 87.10 s. It indicates that the proposal responds to user queries more twice as fast. The fastest response time is observed in *Case 1* and *Case 3*, where the proposed system requires 33.64 s and 17.7 s, respectively, to retrieve articles related to “Web Scraping using Selenium” and “Healthcare Chatbot Natural Language Processing”. It aligns with the small number of available reference papers obtained. In contrast, the proposed system requires a slightly higher response time, where 41.29 s in *Case 2* and 33.8 s in *Case 4*, respectively. It corresponds to the increased the number of available references retrieved, 99 articles in *Case 2* and 85 articles in *Case 4*.

The highest response time occurs in *Case 5*, which requires 48.74 s, corresponding with the second highest number of available reference papers, retrieved up to 87 articles. In contrast, our previous work requires 186.09 s to retrieve a small number of relevant accessible PDFs. We observed an incremental increase in response time load in our previous work when testing all cases.

To measure the efficiency of the response time reduction, we calculated the percentage decrease by the following Equation (3) [56]:

$$\Delta y = \frac{(y_1 - y_2)}{y_1} \times 100\% \quad (3)$$

We assumed that y_1 represents the response time for our previous work, y_2 does the response time for the proposed system, and Δy does the percentage reduction in the response time. The proposal reduces the response time by 59.78% on average, compared to our previous work. It indicates that the proposal achieved good performance using multi-threading with Python *ThreadPoolExecutor* in reducing the response time.

9.3. Performance Comparison with Common Literature Search Tools

The performance comparison results show that the proposal outperforms our previous work. The comparison with common literature search tools is important to understand how the proposal meets practical standards for academic researches in a *reference paper collection system*. We conducted the comparison with common literature search tools such as *Publish or Perish* [24], *OpenAIRE* [25], *OpenAlex* [26], and *Google Scholar* [27]. The performance metrics include the number of collected papers, the number article of *open* and *closed access*, their relevance, PDF accessibility, and the response time. Evaluating the number of *open-access* articles is important, since some platforms may provide *open-access* articles where their PDF files are inaccessible.

9.3.1. Results

Tables 5–9 shows the comparison results.

Table 5. Performance comparison between proposed system and common literature search tools in Case 1.

System	# of Papers	Access Type		Relevance		Accessibility		Response Time (s)
		#of Open A.	#of Closed A.	#of Papers	Rate (%)	#of Papers	Rate (%)	
Publish or Perish [24]	14	7	7	6	42.86%	6	42.86%	4.88 s
OpenAIRE [25]	2	2	0	0	0.00%	0	0.00%	4.31 s
OpenAlex [26]	0	0	0	0	0.00%	0	0.00%	2.67 s
Previous Work [7]	6	6	0	4	66.67%	4	66.67%	27.46 s
Google Scholar [27]	23	12	11	6	26.09%	7	30.43%	0.03 s
Proposal	13	11	2	13	100.00%	13	100.00%	33.64 s

Table 6. Performance comparison between proposed system and common literature search tools in Case 2.

System	# of Papers	Access Type		Relevance		Accessibility		Response Time (s)
		#of Open A.	#of Closed A.	#of Papers	Rate (%)	#of Papers	Rate (%)	
Publish or Perish [24]	100	57	43	31	31.00%	31	31.00%	1.6 s
OpenAIRE [25]	11	6	5	1	9.09%	1	9.09%	3.67 s
OpenAlex [26]	14	7	7	4	28.57%	4	0.00%	2.25 s
Previous Work [7]	8	8	0	8	100.00%	8	100.00%	40.88 s
Google Scholar [27]	4	4	0	4	100.00%	4	100.00%	0.03 s
Proposal	100	97	3	99	99.00%	99	99.00%	41.29 s

Table 7. Performance comparison between proposed system and common literature search tools in Case 3.

System	# of Papers	Access Type		Relevance		Accessibility		Response Time (s)
		#of Open A.	#of Closed A.	#of Papers	Rate (%)	#of Papers	Rate (%)	
Publish or Perish [24]	36	23	13	13	36.11%	13	36.11%	1.15 s
OpenAIRE [25]	5	4	1	0	0.00%	0	0.00%	3.39 s
OpenAlex [26]	0	0	0	0	0.00%	0	0.00%	2.38 s
Previous Work [7]	7	7	0	5	71.43%	5	71.43%	89.02 s
Google Scholar [27]	1	0	1	0	0.00%	0	0.00%	0.04 s
Proposal	66	65	1	53	80.30%	53	80.30%	17.7 s

Table 8. Performance comparison between proposed system and common literature search tools in Case 4.

System	# of Papers	Access Type		Relevance		Accessibility		Response Time (s)
		#of Open A.	#of Closed A.	#of Papers	Rate (%)	#of Papers	Rate (%)	
Publish or Perish [24]	100	65	35	27	27.00%	27	27.00%	2.45 s
OpenAIRE [25]	3	3	0	0	0.00%	0	0.00%	3.64 s
OpenAlex [26]	8	5	3	1	12.50%	1	12.50%	2.2 s
Previous Work [7]	5	5	0	5	100.00%	5	100.00%	92.05 s
Google Scholar [27]	100	77	23	68	68.00%	68	68.00%	0.06 s
Proposal	100	93	7	85	85.00%	99	99.00%	33.8 s

Table 9. Performance comparison between proposed system and common literature search tools in Case 5.

System	# of Papers	Access Type		Relevance		Accessibility		Response Time (s)
		#of Open A.	#of Closed A.	#of Papers	Rate (%)	#of Papers	Rate (%)	
Publish or Perish [24]	14	11	3	6	42.86%	6	42.86%	1.29 s
OpenAIRE [25]	0	0	0	0	0.00%	0	0.00%	2.09 s
OpenAlex [26]	100	81	19	28	28.00%	33	33.00%	1.99 s
Previous Work [7]	2	2	0	2	100.00%	2	100.00%	186.09 s
Google Scholar [27]	100	86	14	77	77.00%	77	77.00%	0.07 s
Proposal	100	93	7	87	87.00%	99	99.00%	48.74 s

9.3.2. Analysis

Tables 5–9 show that Case 2, Case 4, and Case 5 have a higher total number of relevant accessible PDFs compared to Case 1 and Case 3. All the comparisons show the proposed system outperforms others in providing relevant accessible PDFs. In terms of *open access* category, the proposed system outperforms others, but remains competitive with Google Scholar in Case 1, Case 4, and Case 5.

At the same time, *Publish or Perish* provides more *open-access* articles in Case 2 and Case 4 with few relevant accessible PDFs. This also occurred with *OpenAlex*, which found only a few relevant accessible PDFs out of the total *open-access* papers in Case 5. Our previous work [7] provides quite relevant accessible PDFs, although still relatively small in terms of total collected papers.

The lowest response time in all cases achieved by *Google Scholar* was less than 0.1 s. The response time between *Publish or Perish* and *OpenAlex* is less than 3 s. The proposed system has a higher response time than the common literature search tools. Using multi-threading, the proposed system completed the multiple search site tasks ranging from 17.7 to 48.74 s, which is less than 1 min.

9.4. Discussion on Proper Web APIs

The proposed system outperforms in providing relevant accessible PDFs using multiple APIs, compared to our previous work and common literature search tools. By understanding the benefit of multiple API utilization, we further discuss the proper external web APIs to highlight the differences in providing reference papers with accessible PDFs and cover various topics in each case.

Table 10 shows the percentage of each web API in providing accessible PDFs. It suggests that *PubMed* has the highest contribution at 39%, followed by *Semantic Scholar* at 32%, and *DOAJ* has the smallest contribution at 29%.

PubMed provides 42% in Case 2 to the topic “Deep Learning for Skin Detection”, 74.24% in Case 3 to the topic “Healthcare Chatbot Natural Language Processing”, and 49% in Case 5 related to the topic “Large Language Models”. It suggests that *PubMed* primarily covers articles in AI applications with a strong emphasis on the medical domain.

Table 10. Percentage of Web APIs in providing accessible PDFs.

Case(s)	Semantic Scholar	DOAJ	PubMed
	Percentage (%)	Percentage (%)	Percentage (%)
Case 1	84.62%	7.69%	7.69%
Case 2	16%	42%	42%
Case 3	21.21%	4.55%	74.24%
Case 4	25%	54%	21%
Case 5	13%	38%	49%
Average	32%	29%	39%

Semantic Scholar provides 84.62% in *Case 1* to the topic “Web Scraping using Selenium”. It indicates that *Semantic Scholar* effectively provides articles in fields of computer science, especially with technologies and libraries.

DOAJ provides 42% in *Case 2* to the topic “Deep Learning for Skin Detection”, 54% in *Case 4* to the topic “Generative AI for Medical”, and 38% in *Case 5* to the topic “Large Language Models”. This is a broad and general topic, which, with *DOAJ*, covers diverse topics across disciplines.

These findings emphasize the importance of selecting the proper web APIs based on the research focus. However, exploration of other external web APIs such as *Dimensions*, *CrossRef*, *ArXiv*, *BiorXiv*, and *PLOS* [14] have the potential to be integrated into the system to enrich the search results and topic coverage, which will be in our future works.

9.5. Limitations

In this section, we discuss the various limitations and challenges encountered during the development of the *reference paper collection system*. These limitations cover API rate limits and format discrepancies, ethical and legal regulation, scalability and generalization, and robustness and contingency strategies.

9.5.1. API Rate Limits and Format Discrepancies

Several APIs limit the number of requests that can be made in a given time, especially when processing large volumes of data. *Semantic Scholar API* returns up to 1000 papers per request, if more paper matches are found, the API returns a continuation token to continue fetching up to a maximum of 10,000,000 papers [44]. *DOAJ API* imposes a rate limit of two requests per second but allows bursts of up to five requests to be queued to maintain that average and a page size limit of 100 articles per page [45]. *PubMed API* limits 3–10 requests per second with a maximum of 10,000 articles that can be fetched in a single request [46]. The proposed system restricts data retrieval to a maximum of 100 articles per query per API to ensure compliance with rate limits and maintain stable access.

Several APIs return data in different formats, requiring data standardization and integration. For example, *Semantic Scholar* and *DOAJ* APIs return results in *JSON* format, while *PubMed* uses *XML*, requiring different data parsing and transformation techniques. In terms of data quality, *Semantic Scholar* provides fairly complete metadata, but we observed some incomplete field values such as discipline, journal volume, article pages, citations, and PDF URLs. In *DOAJ*, incomplete field values include the number of citations and article pages, although PDF URLs are generally complete due to the high number of *open-access* articles. *PubMed* offers well-organized metadata managed by NCBI, particularly detailed metadata retrieved from *PubMed Central*, including open-access PDF URLs.

9.5.2. Ethical and Legal Regulation

In developing the *reference paper collection system*, we consider each API's ethics and legal regulations. This system uses publicly available *endpoints* provided by *Semantic Scholar*, *DOAJ*, and *PubMed* API. All data retrieval is limited to open-access content and follows each data provider's terms of use and license. The developed system also complies with fair use policies and value restrictions, does not collect personal or sensitive user data, and does not retrieve content that violates copyright or license terms. All information retrieved by the system is used only for academic purposes, thus maintaining ethical integrity in the research process.

Although the three API platforms do not always explicitly mention the prohibition of scraping in the text of their terms of service, we carefully considered at the key points in the API license agreement. *Semantic Scholar* stated at <https://www.semanticscholar.org/product/api/license> [57] to include attribution obtained from "Semantic Scholar" on the website or any publication media, regulates rate limits, suspensions, and use restrictions. *DOAJ* stated at <https://doaj.org/terms> [58], which explains that it provides article metadata openly and freely for use (CC0) and that the metadata can be accessed through various mechanisms provided (webpage, API, OAI-PMH, or data dump). *PubMed* also stated at <https://documentation.uts.nlm.nih.gov/terms-of-service.html> [59], which explains rate limits and facilitates special requests for users who need to request large amounts of data that exceed the request limit.

Respecting all license agreements on each API, we configured the system the developed system by including user agents in the script code, retrieving primary metadata directly from the API *endpoint*, setting a rate limit of 100 articles per API, and setting a delay time between 1 and 3 s for the request. Regarding incomplete PDF URLs in some metadata, we use web scraping or the fast checking of PDF accessibility as an alternative method.

9.5.3. Scalability and Generalization

In terms of scalability, the current system is designed to handle reference search requests on a limited scale and with pre-defined queries. Although multi-threading support and integration of various APIs have been implemented to improve efficiency, the system has not been fully tested for scenarios with large data scales or high concurrent user loads.

In terms of generalization, the capability of the system to generalize various disciplines of scientific domains is still limited. Although our proposed system shows promising results in terms of efficiency and accuracy in the computer science domain through five representative queries, the generalization capabilities to other literature domains with different terminology and keywords have not been thoroughly evaluated. Further work is needed to test the adaptability of the system to a broader range of disciplines, a larger number of queries, and more diverse semantic contexts.

9.5.4. Robustness and Contingency Strategies

To ensure the robustness and contingency strategy of the proposed system, we implemented several techniques to handle the conditions of incomplete data or failures that may occur. As previously described in the multi-thread implementation, when one of the APIs fails to fetch data, the system will return information about which web APIs failed. However, APIs that successfully fetch data will continue to work on the task until it is finished and display the paper list results to the user. For the mechanism to handle incomplete metadata, the system gives an "N/A" mark for incomplete field metadata and applies an alternative method by using fast PDF checking based on *web scraping* for inaccessible PDF URLs.

This mechanism may not be able to fully provide all the data that is expected to be complete, especially in cases where the value in the data source field is empty. Even there is a response failure that cannot be fulfilled by the API platform simultaneously. Therefore, future improvements may involve integrating additional open and legally accessible data sources to enhance data completeness and quality.

10. Conclusions

This paper presented an improved *reference paper collection system* using *web scraping* with three enhancements: (1) integration of multiple APIs, (2) running the program on multiple threads, and (3) fast checking of PDF file accessibility, along with sentence embedding to assess relevance based on semantic similarity. The evaluation demonstrated that the proposal increases the number of available relevant reference papers with full-text PDFs by an average of 64.38% and reduces the response time by an average of 59.78% compared to our previous work, confirming the effectiveness demonstrated in our experiments. The performance comparison with common literature search tools indicated that our system outperforms them in providing higher relevant accessible PDFs and *open-access* articles. Among the external web APIs, *PubMed* has the highest contribution of providing relevant accessible PDF at 39%, followed by *Semantic Scholar* at 32%, and *DOAJ* has the smallest contribution at 29%. However, there are still contributions on meeting user query needs across different literature domains. In future works, we will incorporate *asynchronous I/O* models, such as *asyncio* library, to improve scalability to handle large-scale API queries efficiently. Additionally, we will adapt the system to better understand specific terms in a particular field and user ranking preferences to make its results more relevant to users.

Author Contributions: Conceptualization, T.M.F. and N.F.; methodology, T.M.F.; software, T.M.F.; validation, T.M.F. and N.F.; resources, T.M.F. and I.N.; data curation, T.M.F. and A.M.; writing—original draft preparation, T.M.F.; writing—review and editing, T.M.F., K.C.B., S.T.A. and N.F.; visualization, T.M.F. and D.A.P.; supervision, N.F. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: Data are contained within in the article.

Acknowledgments: We sincerely appreciate all the colleagues in the Distributing System Laboratory, Okayama University who were contributed to this study.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Quaia, E.; Zanon, C.; Vieira, A.; Loewe, C.; Marti-Bonmatí, L. Publishing in open access journals. *Insights Imaging* **2024**, *15*, 212. [\[CrossRef\]](#)
2. Hoy, M.B. Sci-Hub: What librarians should know and do about article piracy. *Med. Ref. Serv. Q.* **2017**, *36*, 73–78. [\[CrossRef\]](#) [\[PubMed\]](#)
3. Segado-Boj, F.; Martín-Quevedo, J.; Prieto-Gutiérrez, J.-J. Jumping over the paywall: Strategies and motivations for scholarly piracy and other alternatives. *Inf. Dev.* **2024**, *40*, 442–460. [\[CrossRef\]](#)
4. Schiermeier, Q. Science publishers try new tack to combat unauthorized paper sharing. *Nature* **2017**, *545*, 145–146. [\[CrossRef\]](#)
5. Pagallo, U.; Ciani, J. Anatomy of web data scraping: Ethics, standards, and the troubles of the law. *Soc. Sci. Res. Netw.* **2023**, *1–12*. [\[CrossRef\]](#)
6. Cao, Y.; Cheung, N.A.; Giustini, D.; LeDue, J.; Murphy, T.H. Scholar Metrics Scraper (SMS): Automated retrieval of citation and author data. *Front. Res. Metrics Anal.* **2024**, *9*, 1335454. [\[CrossRef\]](#)
7. Naing, I.; Aung, S.T.; Wai K.H.; Funabiki, N. A reference paper collection system using web scraping. *Electronics* **2024**, *13*, 2700. [\[CrossRef\]](#)
8. Fricke, S. Semantic Scholar. *J. Med. Libr. Assoc.* **2018**, *106*, 145–147. [\[CrossRef\]](#)

9. Sopan, B.R.; Ramdas, L.S.; Singh, R.N. Directory of Open Access Journals (DOAJ) and its application in academic libraries. *Pearl A J. Libr. Inf. Sci.* **2022**, *16*, 47–56. [CrossRef]
10. Williamson, P.O.; Minter, C.I.J. Exploring PubMed as a reliable resource for scholarly communications services. *J. Med. Libr. Assoc.* **2019**, *107*, 16–29. [CrossRef]
11. Birthare, P.; Raja, M.; Ramachandran, G.; Hargreaves, C.A.; Birthare, S. Covid live multi-threaded live COVID 19 data scraper. In *Structural and Functional Aspects of Biocomputing Systems for Data Processing*; Vignesh, U., Parvathi, R., Goncalves, R., Eds.; IGI Global Scientific Publishing: Hershey, PA, USA, 2023; pp. 28–56. [CrossRef]
12. Pramudita, Y.; Anamisa, D.; Putro, S.; Rahmawanto, M. Extraction system web content sports new based on web crawler multi thread. *J. Phys. Conf. Ser.* **2020**, *1569*, 022077. [CrossRef]
13. Bayly, D.; Kruse, B.; Reidy, C. Streamlit App for Cluster Publication Impact Exploration. In Proceedings of the Practice and Experience in Advanced Research Computing 2024: Human Powered Computing (PEARC '24), New York, NY, USA, 21–25 July 2024. [CrossRef]
14. Velez-Estevez, A.; Perez, I.J.; García-Sánchez, P.; Moral-Munoz, J.A.; Cobo, M.J. New trends in bibliometric APIs: A comparative analysis. *Inf. Process. Manag.* **2023**, *60*, 103385. [CrossRef]
15. Dhalla, H.K. A Performance analysis of native JSON parsers in Java, Python, MS.NET Core, JavaScript, and PHP. In Proceedings of the 2020 16th International Conference on Network and Service Management (CNSM), Izmir, Turkey, 2–6 November 2020. [CrossRef]
16. Fumagalli, S.; Soletta, G.; Agostinetto, G.; Striani, M.; Labra, M.; Casiraghi, M.; Bruno, A. MADAME: A user-friendly bioinformatic tool for data and metadata retrieval in microbiome research. *bioRxiv* **2023**, bioRxiv: 2023.10.14.562335. [CrossRef]
17. Goto, A.; Rodriguez-Esteban, R.; Scharf, S.H.; Morris, G.M. Understanding the genetics of viral drug resistance by integrating clinical data and mining of the scientific literature. *Sci. Rep.* **2022**, *12*, 14476. [CrossRef]
18. Sodian, L.; Wen, J.P.; Davidson, L.; Loskot, P. Concurrency and parallelism in speeding up I/O and CPU-Bound tasks in Python 3.10. In Proceedings of the 2022 2nd International Conference on Computer Science, Electronic Information Engineering and Intelligent Control Technology (CEI), Nanjing, China, 23–25 September 2022. [CrossRef]
19. Thivaharan, S.; Srivatsun, G.; Sarathambekai, S. A survey on Python libraries used for social media content scraping. In Proceedings of the 2020 International Conference on Smart Electronics and Communication (ICOSEC), Trichy, India, 10–12 September 2020. [CrossRef]
20. Adhikari, N.S.; Shradha, A. A comparative study of PDF parsing tools across diverse document categories. *arXiv* **2024**, arXiv:2410.09871. [CrossRef]
21. Stankevičius, L.; Lukoševičius, M. Extracting sentence embeddings from pre-trained transformer models. *Appl. Sci.* **2024**, *14*, 8887. [CrossRef]
22. Jamali, H.R.; Nabavi, M. Open access and sources of full text articles in Google Scholar in different subject fields. *Scientometrics* **2015**, *105*, 1635–1651. [CrossRef]
23. Darmawan, I.; Maulana, M.; Gunawan, R.; Widiyasono, N. Evaluating web scraping performance using XPath, CSS Selector, Regular Expression, and HTML DOM with multiprocessing technical applications. *JOIV Int. J. Inform. Vis.* **2022**, *6*, 904–910. [CrossRef]
24. Publish or Perish. Available online: <https://harzing.com/resources/publish-or-perish> (accessed on 1 November 2024).
25. OpenAIRE. Available online: <https://explore.openaire.eu> (accessed on 1 November 2024).
26. OpenAlex. Available online: <https://openalex.org> (accessed on 1 November 2024).
27. Google Scholar. Available online: <https://scholar.google.com/> (accessed on 1 April 2025).
28. Wu, J.; Kim, K.; Giles, C.L. CiteSeerX: 20 Years of Service to Scholarly Big Data. In Proceedings of the Conference on Artificial Intelligence for Data Discovery and Reuse (AIDR '19), New York, NY, USA, 13–15 May 2019. [CrossRef]
29. Wu, J.; Williams, K.; Chen, H.; Khabsa, M.; Caragea, C.; Tuarob, S.; Ororbia, A.; Jordan, D.; Mitra, P.; Giles, C.L. CiteSeerX: AI in a digital library search engine. *AI Mag.* **2015**, *36*, 35–48. [CrossRef]
30. Knoth, P.; Herrmannova, D.; Cancellieri, M.; Anastasiou, L.; Pontika, N.; Pearce, S.; Gyawali, B.; Pride, D. CORE: A global aggregation service for open access papers. *Sci. Data* **2023**, *10*, 366. [CrossRef]
31. Priem, J.; Piwowar, H.; Orr, R. OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. *arXiv* **2022**, arXiv:2205.0183. [CrossRef]
32. OurResearch GitHub-OpenAlex. Available online: <https://github.com/ourresearch> (accessed on 5 November 2024).
33. Maraver, P.; Armañanzas, R.; Gillette, T.A.; Ascoli, G.A. PaperBot: Open-source web-based search and metadata organization of scientific literature. *BMC Bioinform.* **2019**, *20*, 50. [CrossRef] [PubMed]
34. Atzori, C.; Bardi, A.; Manghi, P.; Mannocci, A. The OpenAIRE workflows for data management. In Proceedings of the Italian Research Conference on Digital Libraries (IRCDL), Modena, Italy, 25 September 2017. [CrossRef]
35. Wojnarski, M.; Kurtz, D.H. Paperity Central: An open catalog of all scholarly literature. *Res. Ideas Outcomes* **2016**, *2*, e8462. [CrossRef]

36. Dande, M.; Narayanan, B.S.L.; Diwadi, A. URL redirect links and its importance calculated using parallel programming to drastically reduced the execution time. *Int. J. Eng. Technol. (UAE)* **2018**, *7*, 95–101. [\[CrossRef\]](#)
37. Du, J.; Liu, Y. An Application Based on Regular Expression. In Proceedings of the International Conference on Computer Networks and Communication Technology (CNCT 2016), Xiamen, China, 16–18 December 2016. [\[CrossRef\]](#)
38. Pant, S.; Yadav, E.N.; Milan; Sharma, M.; Bedi, Y.; Raturi, A. Web scraping using BeautifulSoup. In Proceedings of the 2024 International Conference on Knowledge Engineering and Communication Systems (ICKECS), Chikkaballapur, India, 18–19 April 2024. [\[CrossRef\]](#)
39. Galli, C.; Donos, N.; Calciolari, E. Performance of 4 pre-trained sentence transformer models in the semantic query of a systematic review dataset on peri-implantitis. *Information* **2024**, *15*, 68. [\[CrossRef\]](#)
40. Colangelo, M.T.; Meleti, M.; Guizzard, S.; Calciolari, E.; Galli, C. A comparative analysis of sentence transformer models for automated journal recommendation using PubMed metadata. *Big Data Cogn. Comput.* **2025**, *9*, 67. [\[CrossRef\]](#)
41. Singh, V.K.; Srichandan, S.S.; Piryani, R.; Kanaujia, A.; Bhattacharya, S. Google Scholar as a pointer to open full-text sources of research articles: A useful tool for researchers in regions with poor access to scientific literature. *Afr. J. Sci. Technol. Innov. Dev.* **2022**, *15*, 450–457. [\[CrossRef\]](#)
42. Gupta, V.; Dixit, A.; Sethi, S. A comparative analysis of sentence embedding techniques for document ranking. *J. Web Eng.* **2022**, *21*, 2149–2185. [\[CrossRef\]](#)
43. Kim, H. An investigation of information usefulness of Google Scholar in comparison with Web of Science. *J. Korean BIBLIA Soc. Libr. Inf. Sci.* **2014**, *15*, 215–234. [\[CrossRef\]](#)
44. Semantic Scholar API Documentation. Available online: <https://api.semanticscholar.org/api-docs> (accessed on 7 November 2024).
45. DOAJ API Documentation. Available online: <https://doaj.org/api/v4/docs> (accessed on 7 November 2024).
46. Entrez Programming Utilities Help by National Center for Biotechnology Information. Available online: <https://www.ncbi.nlm.nih.gov/books/NBK25501> (accessed on 7 November 2024).
47. Python Requests for JSON Response. Available online: <https://www.geeksforgeeks.org/response-json-python-requests/> (accessed on 10 November 2024).
48. The Python Standard Library Documentation—ElementTree XML API. Available online: <https://docs.python.org/3/library/xml.etree.elementtree.html> (accessed on 10 November 2024).
49. BeautifulSoup Documentation. Available online: <https://beautiful-soup-4.readthedocs.io/en/latest> (accessed on 12 November 2024).
50. The Python Standard Library Documentation—Regular Expression. Available online: <https://docs.python.org/3/library/re.html> (accessed on 12 November 2024).
51. The Python Standard Library Documentation—Concurrent.futures. Available online: <https://docs.python.org/3/library/concurrent.futures.html> (accessed on 15 November 2024).
52. PyPDF2 Documentation. Available online: <https://pypdf2.readthedocs.io/en/3.x/> (accessed on 1 April 2025).
53. Sentence Transformer Documentation. Available online: <https://www.sbert.net/index.html> (accessed on 1 April 2025).
54. Streamlit.io Documentation. Available online: <https://docs.streamlit.io/get-started> (accessed on 15 November 2024).
55. Aksu, G.; Güzeller, C.O.; Eser, M.T. The effect of the normalization method used in different sample sizes on the success of artificial neural network model. *Int. J. Assess. Tools Educ.* **2019**, *6*, 170–192. [\[CrossRef\]](#)
56. Curran-Everett, D.; Williams, C.L. Explorations in statistics: The analysis of change. *Adv. Physiol. Educ.* **2015**, *39*, 49–54. [\[CrossRef\]](#) [\[PubMed\]](#)
57. Semantic Scholar API License Agreement. Available online: <https://www.semanticscholar.org/product/api/license> (accessed on 1 April 2025).
58. DOAJ API Terms and Conditions. Available online: <https://doaj.org/terms/> (accessed on 1 April 2025).
59. PubMed API Terms of Service. Available online: <https://documentation.uts.nlm.nih.gov/terms-of-service.html> (accessed on 1 April 2025).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.